

INTRODUCCIÓN A STREAMS EN JAVA

¿Qué son los Streams?

Introducidos en Java 8, los Streams permiten realizar operaciones funcionales en colecciones de datos.

Son una secuencia de elementos que pueden ser procesados en paralelo o secuencialmente.

Permiten manipular y transformar datos de manera declarativa y concisa.

CONCEPTOS CLAVE DE STREAMS

Flujo de Datos:

Los Streams representan una secuencia de datos, que pueden ser colecciones, arrays o generados dinámicamente.

Operaciones de Streams:

Operaciones intermedias:

Transforman el Stream (p. ej., filter, map, sorted).

Operaciones terminales:

Producen un resultado (p. ej., collect, forEach, reduce)

CREACIÓN DE UN STREAM

Desde una colección:

```
List<Integer> lista = Arrays.asList(1, 2, 3, 4, 5); Stream<Integer> stream = lista.stream();
```

Desde un array:

```
int[] numeros = {1, 2, 3, 4, 5};  
IntStream stream = Arrays.stream(numeros);
```

Desde valores generados:

```
Stream<String> stream = Stream.of("uno", "dos", "tres");
```


OPERACIONES INTERMEDIAS

`filter()`: Filtra los elementos según una condición.

```
lista.stream().filter(n -> n % 2 == 0).forEach(System.out::println); // Muestra los números
```

`parStream()`: Transforma cada elemento de acuerdo con una función.

```
lista.stream().map(n -> n * 2).forEach(System.out::println); // Multiplica cada número por 2
```

`sorted()`: Ordena los elementos de un Stream.

```
lista.stream().sorted().forEach(System.out::println); // Ordena la lista
```

`distinct()`: Elimina los elementos duplicados.

```
lista.stream().distinct().forEach(System.out::println); // Elimina duplicados
```

OPERACIONES TERMINALES

`forEach()`: Realiza una acción sobre cada elemento.

```
lista.stream().forEach(System.out::println); // Imprime cada elemento
```

`collect()`: Recoge los resultados en una colección.

```
List<Integer> resultado = lista.stream().filter(n -> n % 2 == 0).collect(Collectors.toList());
```

`reduce()`: Acumula los elementos en un solo valor.

```
int suma = lista.stream().reduce(0, (a, b) -> a + b); // Suma todos los elementos
```


OPERACIONES DE REDUCCIÓN

`reduce()`: Acumula los elementos de un Stream en un solo valor.

```
int suma = lista.stream().reduce(0, Integer::sum); // Suma de elementos
```

`count()`: Devuelve el número de elementos.

```
long count = lista.stream().count(); // Cuenta la cantidad de elementos
```

`min()` y `max()`: Obtienen el mínimo o el máximo según un comparador.

```
Optional<Integer> min = lista.stream().min(Integer::compareTo);
```

OPERACIONES CON STREAMS EN PARALELO

`parallelStream()`: Permite procesar los elementos en paralelo.

```
lista.parallelStream().forEach(System.out::println); // Ejecución en paralelo
```

Ventajas:

Mejor rendimiento con grandes volúmenes de datos.

Consideraciones:

Evitar efectos secundarios al usar operaciones en paralelo.

USO COMÚN DE STREAMS: FILTRAR Y TRANSFORMAR

Filtrar elementos: Usamos `filter()` para seleccionar elementos que cumplan una condición.

```
lista.stream().filter(n -> n > 2).forEach(System.out::println); // Filtrar números mayores que 2
```

Transformar elementos: Usamos `map()` para aplicar una función a cada elemento.

```
lista.stream().map(n -> n * 2).forEach(System.out::println); // Duplicar cada elemento
```


USO DE COLLECT() CON STREAMS

Colección en una lista:

```
List<Integer> listaFiltrada = lista.stream()  
    .filter(n -> n % 2 == 0)  
    .collect(Collectors.toList());
```

Colección en un Set:

```
Set<Integer> setFiltrado = lista.stream()  
    .filter(n -> n % 2 == 0)  
    .collect(Collectors.toSet());
```

Unir elementos:

```
String resultado = lista.stream()  
    .map(String::valueOf)  
    .collect(Collectors.joining(", ", "[", "]"));
```

USO DE OPTIONAL CON STREAMS

Optional: Representa un valor que puede ser nulo o presente.

```
Optional<Integer> max = lista.stream().max(Integer::compareTo);
```

Evitar NullPointerException:

```
max.ifPresent(System.out::println); // Si hay valor, lo imprime
```

BUENAS PRÁCTICAS CON STREAMS

Evitar efectos secundarios:

Los Streams son diseñados para ser declarativos y no mutar el estado.

Operaciones perezosas:

Las operaciones intermedias no se ejecutan hasta que se llama a una operación terminal.

Reutilización de Streams:

Un Stream solo se puede consumir una vez. Si necesitas reutilizarlo, crea un nuevo Stream.

CONCLUSIÓN SOBRE STREAMS EN JAVA

Poderosos y flexibles:

Permiten procesar colecciones de manera eficiente y declarativa.

Composición de operaciones:

Puedes encadenar múltiples operaciones de manera fluida.

Paralelismo:

Ofrecen un modelo sencillo para la ejecución en paralelo, mejorando el rendimiento.